

Revisión funcional del carrito de Frutería Python

Qué conviene corregir antes de cerrar el ejercicio

Procesos revisados	Criterio
Agregar · Ver · Actualizar · Eliminar · Finalizar · Cancelar	Prioridad a reglas de negocio, coherencia del carrito, navegación y recuperación ante cancelaciones.

Objetivo: mejorar el comportamiento típico de una compra de frutería sin convertir el ejercicio en un sistema comercial completo.

1. Reglas de negocio que deben quedar definidas

Antes de tocar cada opción del menú, estas reglas evitan inconsistencias entre pantallas.

- **Peso:** vender por gramos, por tanto aceptar como máximo 3 decimales en kg (ej.: 1,250 kg).
- **Dinero:** mostrar y calcular importes comerciales con 2 decimales.
- **Compra por importe:** calcular el peso a 3 decimales y después recalcular el subtotal a 2 decimales para que peso x precio sea coherente.
- **Códigos:** aceptar tanto 1 como 01, normalizando internamente a dos dígitos.
- **Navegación:** ofrecer Volver/Cancelar en las selecciones donde el usuario pueda quedar atrapado.
- **Conceptos adicionales:** mantener una regla clara para distinguir fruta, bolsa y envío.
- **Bolsas:** usar un único código, por ejemplo B01, tanto para bolsas manuales como automáticas.

Mejora técnica recomendable, pero no imprescindible para cerrar el ejercicio: investigar Decimal para dinero en una versión posterior.

2. Agregar producto

Es el proceso con más validaciones de entrada y donde conviene reforzar la experiencia de compra.

- **Buscar por nombre:** si hay coincidencia parcial, mostrarla y permitir confirmar o elegir. Si la opción introducida es inválida, explicar el error y volver a preguntar.
- **Productos agotados:** idealmente no ofrecerlos como una elección válida o avisar antes de entrar en la selección.
- **Compra por peso:** impedir cantidades con más de 3 decimales y cantidades absurdamente grandes.
- **Compra por euros:** después de convertir euros a kg, recalcular el subtotal usando el peso redondeado.
- **Producto ya existente:** se puede seguir fusionando cantidades, pero sería más claro mostrar cantidad anterior, cantidad añadida y nuevo total.
- **Compra por unidad:** permitir seleccionar y también desmarcar una pieza antes de terminar.
- **Unidades disponibles:** volver a mostrar la lista tras cada selección/desmarcado e incluir opciones visibles para terminar o cancelar.
- **Presupuesto:** mostrar el disponible antes de elegir puede ayudar; como mínimo mantener la validación antes de incorporar el producto.

Resultado esperado: el cliente puede buscar, corregirse, volver atrás y conocer exactamente cuánto peso o qué piezas está incorporando.

3. Ver carrito

Debe funcionar como el estado fiable de la compra: lo que el cliente ve debe coincidir con lo que se cobrará.

- **Presupuesto:** si se supera, no limitarse a mostrar 'Resta: -X €'; indicar claramente 'Presupuesto excedido en X €'.
- **Bolsas automáticas:** si una finalización a domicilio se cancela, no deben quedar bolsas residuales que ya no correspondan al estado elegido.
- **Conceptos:** diferenciar visualmente productos, bolsas y envío. El envío puede seguir fuera de la tabla, pero debe verse antes de confirmar.
- **Total:** subtotal, bolsas, envío y total deben seguir la misma regla de redondeo.
- **Formato:** los emojis pueden desalinear columnas; priorizar legibilidad aunque la tabla sea menos decorativa.
- **Navegación:** después de consultar el carrito, volver claramente al menú principal.

Regla funcional: el carrito es la 'fuente visual de verdad' para el cliente antes de finalizar.

4. Actualizar producto

La actualización actual funciona, pero temporalmente devuelve el stock anterior y por eso requiere especial cuidado.

- **Mostrar solo productos del carrito:** es más natural que buscar en todo el catálogo y después descubrir que el producto no estaba comprado.
- **Estado anterior:** mostrar cantidad/piezas actuales antes de pedir la nueva selección.
- **Cancelación segura:** si el usuario cancela o supera presupuesto, el carrito y el stock deben quedar exactamente como estaban antes.
- **Productos por unidad:** mostrar qué piezas tenía seleccionadas y cuáles vuelven a estar disponibles durante la modificación.
- **Tipo de actualización:** para este ejercicio puede mantenerse 'sustituir toda la selección'. Añadir/reducir parcialmente sería una mejora futura.
- **Bolsas:** recalcularlas tras actualizar y, si cambia su cantidad, informar al usuario.
- **Presupuesto:** validar la nueva selección antes de hacer definitivo el cambio.

Invariante clave para depuración: una actualización cancelada nunca debe alterar ni el carrito ni el stock.

5. Eliminar producto

Es un flujo sencillo; las mejoras principales son confirmación, comunicación y coherencia de stock.

- **Mostrar productos del carrito:** evitar que el usuario busque frutas que no ha comprado.
- **Confirmación:** mantener S/N antes de eliminar.
- **Respuesta N:** mostrar 'Eliminación cancelada' para que el usuario sepa qué ocurrió.
- **Stock:** devolver correctamente kg o piezas antes de borrar la línea.
- **Bolsas:** recalcularlas después de eliminar y avisar si se reduce el número necesario.
- **Eliminación parcial:** no es imprescindible ahora; puede quedar como mejora futura porque Actualizar ya permite sustituir la cantidad completa.

Resultado esperado: eliminar una línea debe dejar stock, carrito, bolsas y total sincronizados.

6. Finalizar compra

Es el punto de mayor prioridad porque aquí se consolidan entrega, bolsas, presupuesto y confirmación.

- **Mantener visible el carrito:** al entrar en Finalizar, mostrar el resumen actual antes o junto a la elección de entrega.
- **Entrega online:** elegir Recogida o Domicilio y mostrar inmediatamente su efecto en el total.
- **Domicilio:** recalcular bolsas automáticas según el peso y añadir el coste de envío antes de confirmar.
- **Recogida/tienda:** explicar que 0 bolsas significa 'ninguna'.
- **Presupuesto:** volver a validarlo con bolsas y envío incluidos. Si se supera, explicar cuánto se excede.
- **Si se excede:** permitir volver al carrito; como mejora, ofrecer cambiar a recogida o quitar productos.
- **Confirmación final:** si el cliente responde N, decidir explícitamente si la elección de entrega y las bolsas se conservan o se revierten. Para un flujo limpio, conviene revertir cambios temporales de finalización.
- **Ticket:** solo debe generarse después de una confirmación S.

Regla recomendada: entrar en Finalizar no debe modificar de forma irreversible el pedido hasta que el cliente confirme.

7. Cancelar compra

Es uno de los procesos más sólidos del programa y conviene conservar su simplicidad.

- **Confirmación:** preguntar antes de cancelar toda la compra.
- **Restaurar stock:** devolver todas las frutas y piezas reservadas.
- **Vaciar carrito:** eliminar también bolsas y cualquier concepto adicional asociado al pedido.
- **Mensaje final:** confirmar que la compra fue cancelada y el stock restaurado.
- **Ctrl+C:** hoy el stock está solo en memoria, pero si más adelante existe persistencia deberá decidirse si una interrupción equivale a cancelar o a conservar el pedido.
- **Diseño futuro:** evitar que cancelar dependa de comprobar nombres especiales como 'Bolsa'; distinguir los elementos por un campo de tipo.

Para la versión actual: no añadir complejidad innecesaria; asegurar que cancelar siempre deja el programa en un estado coherente.

8. Ticket, bolsas y envío

Aunque no son opciones principales del menú, afectan directamente a Ver, Finalizar y Cancelar.

- **Código de bolsa:** unificar B1/B01.
- **Envío:** mostrarlo claramente antes de confirmar y también en el ticket.
- **Redondeo:** peso a 3 decimales; dinero a 2. Mantener la misma regla en línea, subtotal y total.
- **Ticket:** corregir o simplificar el formato si los emojis desalinean las columnas.
- **Identificador:** el código basado en segundos puede repetirse en casos extremos; para el ejercicio es aceptable, pero debe anotarse como limitación.
- **Persistencia:** guardar ticket, histórico y stock en archivo/base de datos es una fase posterior, no una corrección imprescindible del flujo actual.

No introducir todavía IVA, promociones, cupones, facturación o pago salvo que formen parte explícita de los requisitos del ejercicio.

9. Orden recomendado de corrección

Primero lo que afecta al negocio y al estado del pedido; después lo puramente visual.

Prioridad	Corrección	Procesos afectados
1	Definir peso a 3 decimales y dinero a 2	Agregar · Ver · Finalizar · Ticket
2	Corregir compra por importe para que peso/precio/subtotal coincidan	Agregar
3	Mejorar búsqueda: confirmación, reintento y código 1/01	Agregar · Actualizar · Eliminar
4	Permitir desmarcar unidades y mejorar Volver/Terminar	Agregar · Actualizar
5	Mantener carrito visible y controlar cancelación de la finalización	Ver · Finalizar
6	Garantizar restauración exacta de stock al actualizar/eliminar/cancelar	Actualizar · Eliminar · Cancelar
7	Unificar bolsas y recalcularlas de forma coherente	Agregar · Ver · Actualizar · Eliminar · Finalizar
8	Mensajes claros de cancelación y presupuesto excedido	Ver · Actualizar · Eliminar · Finalizar
9	Ajustar ticket y alineación	Ticket

10. Criterio de cierre

Qué debería quedar resuelto y qué puede documentarse como mejora futura.

Debe quedar resuelto

- Búsqueda y navegación comprensibles.
- Peso máximo 3 decimales.
- Dinero a 2 decimales.
- Compra por importe coherente.
- Selección por unidades reversible.
- Stock coherente en actualizar/eliminar/cancelar.
- Carrito visible antes de confirmar.
- Presupuesto final con bolsas y envío.
- Bolsas con código/regla única.

Puede quedar para una versión futura

- Decimal para toda la contabilidad.
- Persistencia de stock.
- Histórico de pedidos/tickets.
- Pago y fallo de pago.
- Dirección, teléfono y franja horaria.
- IVA, descuentos y promociones.
- Identificador robusto por pieza.
- Pedido mínimo, zonas y límites de reparto.

Meta final: que cada operación pueda cancelarse o fallar sin dejar carrito, stock, bolsas, presupuesto o total en un estado incoherente.